Dockerized Web Application

Module 4 - Docker & Containerization (Day 16 - Day 20)

Project Documentation

Prepared by: **<Your Name>** | Intern, <Company Name>
GitHub: github.com/<your-username>/<your-repo>

1. Overview

This project containerizes a web application using Docker so that it runs consistently across any environment, eliminating "works on my machine" issues. The application is orchestrated with Docker Compose and demonstrates the complete workflow from source code to a running application.

Source Code > **Dockerfile** > **Image** > **Container** > **Docker Compose** > **Running App**

- Write an optimized **Dockerfile**
- Build and run **images and containers**
- Manage **networking, volumes and persistent data**
- Configure the app with **environment variables**
- Orchestrate services with **Docker Compose**

2. Containers vs Virtual Machines

Feature	Containers	Virtual Machines
Isolation	Process-level	Full OS-level
Startup time	Seconds	Minutes
Size	Lightweight (MBs)	Heavy (GBs)
Resource usage	Shares host OS kernel	Dedicated guest OS
Portability	Highly portable	Less portable
Performance	Near-native	Hypervisor overhead

3. Docker Architecture

- **Docker Client** - CLI used to issue commands (docker build, docker run).
- **Docker Daemon (dockerd)** - builds, runs and manages containers.

- **Docker Images** - read-only templates used to create containers.
- **Docker Containers** - running instances of images.
- **Docker Registry** - storage for images (e.g., Docker Hub).

4. Tech Stack

Layer	Technology
Application	<Node.js / Python / Java / etc.>
Containerization	Docker
Orchestration	Docker Compose
Version Control	Git & GitHub

5. Project Structure

```
.
|-- app/                  # Application source code
|   |-- src/
|   `-- package.json
|-- Dockerfile            # Image build instructions
|-- docker-compose.yml    # Multi-container configuration
|-- .dockerignore         # Files excluded from build context
|-- .env.example          # Sample environment variables
`-- README.md
```

6. Dockerfile

```
# Lightweight base image
FROM node:20-alpine

# Set working directory
WORKDIR /app

# Copy dependency files first (layer caching)
COPY package*.json ./
RUN npm ci --omit=dev

# Copy the rest of the source code
COPY . .

# Expose application port
EXPOSE 3000

# Start the application
CMD ["npm", "start"]
```

7. Docker Compose

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    env_file:
      - .env
    volumes:
      - app-data:/app/data
    networks:
      - app-network
    restart: unless-stopped

volumes:
  app-data:

networks:
  app-network:
    driver: bridge
```

8. Getting Started

Prerequisites: Docker, Docker Compose and Git installed.

```
# 1. Clone the repository
git clone https://github.com/<your-username>/<your-repo>.git
cd <your-repo>

# 2. Create the environment file
cp .env.example .env

# 3. Build and start the application
docker compose up --build -d

# 4. Open in browser:  http://localhost:3000

# Stop the application
docker compose down
```

9. Essential Docker Commands

Command	Description
`docker build -t myapp .`	Build an image from a Dockerfile
`docker images`	List local images
`docker run -d -p 3000:3000 myapp`	Run a container in detached mode
`docker ps / docker ps -a`	List running / all containers
`docker logs <container>`	View container logs
`docker exec -it <container> sh`	Open a shell inside a container
`docker stop <container>`	Stop a container

Command	Description
`docker rm <container>`	Remove a container
`docker rmi <image>`	Remove an image
`docker compose up -d`	Start all services
`docker compose down`	Stop and remove all services

10. Networking, Volumes & Environment Variables

- **Networking** - containers on the same user-defined bridge network communicate using service names (e.g., http://web:3000).
- **Volumes** - named volumes persist data beyond the container lifecycle, so data survives restarts and removals.
- **Environment Variables** - configuration is injected at runtime via a .env file, keeping secrets out of the image (e.g., PORT=3000, NODE_ENV=production).

11. Dockerfile Optimization

- Used a **minimal base image** (Alpine) to reduce image size.
- Copied **dependency files first** to maximize layer caching.
- Added a **.dockerignore** to exclude node_modules, .git and logs.
- Installed **only production dependencies**.
- Combined commands to **reduce the number of layers**.

12. Key Learnings

- Containers are lightweight, fast and portable compared to virtual machines.
- Understanding images, layers and caching leads to faster builds.
- Volumes are essential for persistent data.
- Docker Compose simplifies running multi-container applications with a single command.
- Environment variables keep configuration flexible and secure.